

架构候选方案对比

团队： 暴风星云裂 | 项目： CampusHub | 日期： 2026年4月29日

1. 项目背景与约束

CampusHub 是面向南京大学在校学生的轻量校园互助服务平台，核心流程是“发布需求—接单确认—沟通执行—确认完成—评价信用—举报审核”。P1 需求文档已经明确本项目是 10 周内交付的学生 MVP，不追求商业级复杂功能。

本阶段架构选择必须满足以下约束：

- **团队约束：** 3 人学生团队，人均每周约 6 小时，P2 阶段由王自宽负责架构设计。
- **功能范围：** 用户认证、需求发布、任务大厅、订单状态流转、订单内实时文字和图片聊天、评价信用、举报审核、基础后台。
- **技术约束：** 前端 Vue3，后端 Java，框架 Spring Boot，数据库 MySQL。
- **成本约束：** 使用开源技术和免费/低成本服务器，不引入需要专业运维的组件。
- **实现边界：** 不实现在线支付、推荐算法、语音/视频通话、群聊、位置共享、复杂数据看板、自动化内容审核。
- **非功能目标：** 支持约 200 名同时在线用户，核心操作响应时间小于 2 秒，页面加载小于 4 秒。

2. 候选架构方案

方案 A：前后端分离的分层单体架构

前端使用 Vue3 构建单页应用，后端使用一个 Spring Boot 应用承载业务模块，内部按 Controller、Service、Repository/Mapper 分层。各业务模块在同一代码库、同一进程内通过服务接口协作，统一访问 MySQL。

该方案本质上结合了：

- **前后端分离：** 浏览器端 Vue3 与后端 RESTful API 解耦。
- **MVC/分层架构：** 后端使用 Spring MVC，对表现层、业务层、数据访问层分责。
- **模块化单体：** 用户、需求、订单、消息、评价、举报、后台等按包和接口划分边界，但不拆成多个服务。

方案 B：微服务架构

将系统拆为用户服务、需求服务、订单服务、消息服务、评价服务、审核服务、后台服务等多个独立服务。每个服务独立部署，服务间通过 HTTP/RPC 或消息队列通信，可按业务独立扩容。

方案 C：事件驱动架构

围绕“需求发布”“接单成功”“订单完成”“评价提交”“举报处理”等业务事件设计系统。服务之间通过事件总线或消息队列异步通信，适合通知、审计、积分等副作用处理。

3. 多维度对比

对比维度	方案 A：前后端分离的分层单体	方案 B：微服务	方案 C：事件驱动
开发复杂度	低到中。一个 Spring Boot 应用即可完成主要业务，团队只需维护一套后端工程。	高。需要服务拆分、服务间接口、认证传递、部署编排、日志追踪。	中到高。需要事件建模、消息可靠投递、重复消费处理和失败补偿。
可维护性	高。模块边界清晰，代码集中，适合课程项目迭代。	理论上高，但小团队容易出现接口漂移和跨服务调试困难。	对事件副作用维护友好，但主业务链路会变得不直观。
可扩展性	中。可先通过模块边界保留后续拆分空间。	高。适合用户量大、团队多人并行维护的大型系统。	高。适合通知、审计、积分等异步场景扩展。
性能	足够。200 并发目标可通过单体应用、MySQL 索引、分页查询满足。	单服务性能可扩展，但服务间网络调用增加延迟。	异步场景性能好，但核心订单状态仍需同步事务保障。
数据一致性	容易保障。订单、信用、通知等关键更新可使用本地事务。	较难。跨服务事务需要最终一致性或补偿机制。	较难。事件重复、乱序、失败重试都要额外处理。
团队学习成本	低。Spring Boot + Vue3 + MySQL 都是主流且资料丰富。	高。需要学习服务治理、网关、配置中心、链路追踪等。	中到高。需要学习消息队列、幂等、事务消息等。
运维成本	低。单机部署即可：Nginx + Spring Boot Jar + MySQL。	高。多个服务部署和监控成本明显上升。	中到高。通常需要 RabbitMQ/Kafka/Redis Stream 等中间件。
10 周内可行性	高。最符合 MVP 和课程项目节奏。	低。大量时间会花在基础设施而不是业务闭环。	中低。适合后续优化，不适合作为首版主架构。

4. 结论

最终选择 **方案 A：前后端分离的分层单体架构**。

选择理由：

- 最符合 P1 需求边界。** CampusHub 的核心复杂度在订单状态流转、权限校验和举报处理，而不是海量并发或多团队协作。
- 最适合学生项目交付。** Vue3 + Spring Boot + MySQL 是主流、稳定、资料丰富的组合，能把时间集中在业务闭环和测试上。
- 能减少不必要中间件。** MVP 阶段不使用 Redis、消息队列、网关、搜索引擎等组件；订单内聊天使用 Spring WebSocket + MySQL 消息表实现，系统通知可用数据库表 + HTTP 轮询实现，定时任务用 Spring Task 实现。
- 保留后续演进空间。** 后端仍按模块划分接口，未来若用户规模扩大，可优先拆分文件服务、通知服务或后台审核服务。

不选择其他方案的原因：

- **不选择微服务：** 当前团队规模和并发目标不需要微服务。认证、订单、消息、审核服务之间存在频繁协作，拆分后需要大量接口编排，举报处理和订单争议往往需要跨服务查询上下文，调试和追踪复杂，当前并发目标只有 200 人，尚未达到必须通过服务拆分解决容量问题的程度。
- **不选择事件驱动作为主架构：** 订单状态流转需要强一致和可追踪，事件驱动更适合作为后续通知、审计、异步任务的局部优化，而不是首版主干。
- **不选择纯前后端混合 MVC：** 前后端分离更适合 Vue3 页面开发、接口测试和后续移动端 H5 适配，也便于团队分工。